

Codebase Analysis Report: Frontend

Generated: 2026-06-03 16:44:31

Analyzed by: Codebase Analysis Agent (Claude AI / Anthropic)

Root Path: C:\Users\2402013\OneDrive - Cognizant\Desktop\Frontend

Table of Contents

- [1. Executive Summary](#)
- [2. Project Overview](#)
- [3. Technical Architecture](#)
- [4. Functional Flow](#)
- [5. Data Flow Analysis](#)
- [6. Component Relationships](#)
- [7. File Inventory](#)
- [8. API Endpoints](#)
- [9. Dependencies](#)
- [10. Developer Onboarding Guide](#)

1. Executive Summary

Executive Summary -- Frontend (compliancetrack-frontend)

The Frontend project is the client-facing Single-Page Application (SPA) for a compliance and audit tracking platform. Built in React with a mix of JavaScript, TypeScript, and TSX (50 files, ~6,700 lines), it provides the user interface for managing compliance workflows, audits, and related analytics. The application communicates with an external backend through a centralized `apiClient`; that backend is not part of this codebase, making this repository purely the presentation and client-side logic layer of the broader system.

Architecturally, the project adopts a Feature-Sliced / Layered SPA pattern that combines horizontal layering with vertical feature isolation. Domain functionality is organized under `src/features/*`, where self-contained slices such as `analytics` encapsulate their own components, state, and logic. This separation keeps cross-feature coupling low and allows individual domains to evolve independently, while shared concerns (such as the API client and common UI primitives) live in dedicated horizontal layers consumed across features.

The codebase's notable strengths lie in its disciplined modular boundaries and its centralized backend access through `apiClient`, which isolates network concerns from feature code and provides a single point for handling authentication, error handling, and request configuration. The feature-sliced structure scales well as new compliance domains are added, and the incremental adoption of TypeScript/TSX alongside JavaScript indicates an ongoing migration toward stronger type safety. With 18 classes and 183 functions distributed across the slices, logic appears reasonably decomposed rather than concentrated in monolithic modules.

A new developer should begin by orienting around the `src/features` directory to understand the domain boundaries, then study the `apiClient` abstraction to learn how the frontend integrates with the external backend and where data contracts are defined. Because no explicit entry point was detected, an early priority should be locating the application bootstrap (typically the root render and routing configuration) to map how features are composed into the running SPA. Finally, given the mixed-language nature of the codebase, contributors should note which areas are still plain JavaScript versus typed TypeScript, as this affects refactoring safety and onboarding effort.

2. Project Overview

Project Statistics

Metric	Value
Total Files Analyzed	50
Total Classes	18
Total Functions	183
Total API Endpoints Detected	0
Total Lines of Code	6,736

Language Breakdown

Language	Files
TSX	38
TypeScript	11
JavaScript	1

Entry Points

No entry points auto-detected.

Project Structure

```
Frontend/
+-- compliancetrack-frontend
|   +-- public
|   |   +-- vite.svg
|   +-- src
|   |   +-- app
|   |   |   +-- layouts
|   |   |   +-- routes
|   |   |   +-- store
|   |   +-- features
|   |   |   +-- analytics
|   |   |   +-- audit-task-checklist
|   |   |   +-- auditTask
|   |   |   +-- auth
|   |   |   +-- compliance
|   |   |   +-- employee-type
|   |   |   +-- master-data
|   |   |   +-- my-policies
|   |   |   +-- notifications
|   |   |   +-- policy
|   |   |   +-- policyAssessment
|   |   |   +-- reports
|   |   |   +-- user-dashboard
|   |   |   +-- user-management
|   |   +-- shared
|   |   |   +-- api
|   |   |   +-- components
|   |   |   +-- constants
|   |   |   +-- hooks
|   |   |   +-- types
|   |   |   +-- ui
|   |   |   +-- utils
|   |   +-- App.css
|   |   +-- App.tsx
|   |   +-- index.css
|   |   +-- main.tsx
|   +-- AUTHENTICATION_FLOW.md
|   +-- eslint.config.js
|   +-- index.html
|   +-- package-lock.json
|   +-- package.json
|   +-- README.md
|   +-- tsconfig.app.json
|   +-- tsconfig.json
|   +-- tsconfig.node.json
|   +-- vite.config.ts
+-- package-lock.json
```

3. Technical Architecture

Technical Architecture Summary -- Project "Frontend"

Scope note: The analysis covers the `compliance-track-frontend` React application. This is a **Single-Page Application (SPA) frontend** for a compliance/audit tracking platform. The backend referenced via `apiClient` is external and not included in this codebase.

1. Architecture Pattern

The application follows a **Feature-Sliced / Layered SPA architecture** with clear horizontal layering and vertical feature isolation. Evidence:

- **Feature-based vertical slicing:** `src/features/*` contains self-contained domains (`analytics`, `auditTask`, `auth`, `compliance`, `policy`, `policyAssessment`, `reports`, `user-management`, etc.). Each feature bundles its own UI (`*Page.tsx`), API calls (`*Api.ts`), and hooks (`hooks/use*.ts`).
- **Shared horizontal layers:** `src/shared/*` provides cross-cutting concerns split into `api`, `components`, `constants`, `hooks`, `types`, `ui`, `utils`.
- **Application shell layer:** `src/app/*` holds routing (`routes/`), layouts (`layouts/`), and global state (`store/`).

A **container/presentational separation** is visible within features. For example, in analytics:

- `DashboardPage.tsx` (container -- orchestrates data via `useAnalyticsData`, handles save logic)
- `useAnalyticsData.ts` (data-fetching/state hook)
- `AdminCharts.tsx`, `ManagerCharts.tsx`, `SharedCharts.tsx`, `ChartComponents.tsx` (presentational components receiving props)
- `analyticsApi.ts` (API/service layer)

This is a **Layered + Feature-Sliced** pattern, not MVC/microservice/event-driven.

2. Technology Stack

Layer	Technology	Evidence
Language	TypeScript (TSX/TS), minimal JS	38 TSX, 11 TS, 1 JS file; <code>tsconfig.*.json</code> present
UI Framework	React (with <code>lazy/Suspense</code>)	<code>import { lazy, Suspense } from "react" in AppRouter.tsx</code>
Build Tool	Vite	<code>vite.config.ts</code> , <code>public/vite.svg</code> , <code>reactRefresh.configs.vite</code>

Routing	React Router (v7 module <code>react-router</code>)	<code>import { BrowserRouter, Routes, Route } from "react-router"</code>
State Management	Redux Toolkit	<code>configureStore, createSlice</code> in <code>store.ts</code> , <code>authSlice.ts</code>
Charting	Recharts	<code>PieChart, BarChart, AreaChart, LineChart</code> imports across chart components
Notifications/Toasts	Sonner	<code>import { Toaster } from "sonner", toast.error(...)</code>
Icons	Lucide React	<code>import { TrendingUp, Clock, ... } from "lucide-react"</code>
HTTP Client	Axios (wrapped)	<code>apiClient.get<...></code> typed generics in <code>analyticsApi.ts</code>
Styling	Bootstrap (CSS classes) + custom CSS	<code>className="card border-0 shadow-sm", d-flex, App.css, dashboard.css</code>
Linting	ESLint (flat config) + typescript-eslint	<code>eslint.config.js</code> with <code>tseslint</code> , <code>reactHooks</code> , <code>reactRefresh</code>
Language Target	ECMAScript 2020	<code>ecmaVersion: 2020</code> in ESLint config

3. Module / Layer Responsibilities

`src/app/` -- Application Shell

- `routes/AppRouter.tsx`: Central routing table. Uses `lazy()` for code-splitting every page. Performs **role-based root redirect** (`EMPLOYEE` -> `MY_POLICIES`, others -> `DASHBOARD`) reading from Redux auth state.
- `routes/PrivateRoute.tsx`: Route guard component. Checks `token`, `user`, and `user.role`; supports `allowedRoles` prop for **RBAC** (roles `ADMIN` | `MANAGER` | `EMPLOYEE`).

- `layouts/MainLayout.tsx`: Authenticated shell -- `Sidebar` + `Header` + `<Outlet/>`. Shared across all roles.
- `store/store.ts`: Redux store configuration; exposes `RootState` and `AppDispatch` types.
- `store/authSlice.ts`: Auth state slice with `setAuth/clearAuth` reducers plus `localStorage` persistence helpers (`loadAuthFromStorage`, `saveAuthToStorage`, `clearAuthStorage`).

`src/features/` -- **Domain Features**

Each folder is a bounded vertical slice. Notable structure in `analytics`:

- `DashboardPage.tsx` -- current dashboard container (role-aware: `isAdmin`, `isManager`).
- `DashboardOld.tsx` / `DashboardDemo.tsx` -- legacy/demo versions (technical debt: superseded files retained).
- `analyticsApi.ts` -- 12 typed endpoint functions (e.g., `getDashboardSummary`, `getDepartmentCompliance`, `getTeamPendingPolicies`), organized by access level (`GENERAL`, `ADMIN ONLY`, `ADMIN & MANAGER`).
- `hooks/useAnalyticsData.ts` -- central data-fetching hook with `loadSafely` resilience pattern and per-chart reload functions.
- `components/` -- `AdminCharts`, `ManagerCharts`, `SharedCharts`, `ChartComponents`, `DashboardHeader` (pure presentational chart components).

Other feature areas (`auditTask/{admin,manager,employee}`, `policyAssessment/ManagerAndEmployeeQuiz`, `compliance`, `reports`, `my-policies`) follow the same role-segmented page convention.

`src/shared/` -- **Cross-Cutting Concerns**

- `api/apiClient` -- central HTTP client (Axios wrapper).
- `constants/` -- `routes.ts` (`ROUTES`), `apiEndpoints.ts` (`API_ENDPOINTS`).
- `types/` -- shared TS interfaces (`auth.ts`, `analytics.ts`, `report.ts`).
- `ui/` -- generic presentational components (`LoadingSpinner`, `PlaceholderPage`, `NotFoundPage`).
- `components/` -- `Sidebar`, `Header`.
- `hooks/` -- `useTheme` (global theme initialization, called in `App.tsx`).
- `utils/` -- `errorParser.ts` (`parseError`), `dateFormatter.ts` (`toISOString`), `formatNumber.ts`.

4. External Dependencies & Integrations

Integration	Type	Evidence
Backend REST API	HTTP/REST	<code>apiClient.get<DashboardSummary>(…)</code> with <code>API_ENDPOINTS.*</code> constants

Browser localStorage	Client persistence	<code>localStorage.getItem/setItem/removeItem</code> in <code>authSlice.ts</code>
Authentication service	Token-based auth (external)	<code>state.auth.token</code> , <code>LoginResponse</code> , <code>LoginPage</code> , <code>RegisterPage</code> , <code>AUTHENTICATION_FLOW.md</code>

API endpoints observed (analytics domain): `ANALYTICS_SUMMARY`, `ANALYTICS_MOST_ASSIGNED_POLICIES`, `ANALYTICS_POLICIES_BY_CATEGORY`, `ANALYTICS_COMPLIANCE_DEPARTMENT`, `ANALYTICS_MONTHLY_ROLLOUT`, `ANALYTICS_CHECKLIST_BUBBLE`, `ANALYTICS_AVG_QUIZ_SCORE`. Query parameters are appended manually (`?userId=`, `URLSearchParams`).

No databases, message queues, or GraphQL clients are present in the frontend codebase.

5. Design Patterns (with evidence)

Pattern	Evidence
Flux/Redux Store (Single Source of Truth)	<code>configureStore({ reducer: { auth } })</code> ; components read via <code>useSelector((state: RootState) => state.auth.token)</code>
Slice / Reducer pattern	<code>createSlice({ name: "auth", reducers: { setAuth, clearAuth } })</code>
Route Guard / Decorator	<code>PrivateRoute</code> wraps protected routes, enforcing auth + <code>allowedRoles</code>
Container-Presentational	<code>DashboardPage</code> (logic) vs <code>AdminCharts/SharedCharts</code> (props-only rendering)
Custom Hook (Facade over data)	<code>useAnalyticsData</code> aggregates 14 API calls into one state object + reload functions
Lazy Loading / Code Splitting	<code>lazy(() => import(...))</code> for every routed page, wrapped in <code><Suspense fallback={<LoadingFallback/>}></code>
Resilience / Graceful Degradation	<code>loadSafely</code> wrapper + per-chart <code>errors: Record<string, string></code> ; <code>ErrorCard/EmptyCard</code> render per failing chart so one failure doesn't break the dashboard

Singleton (module instance)	Shared <code>apiClient</code> module imported across all <code>*Api.ts</code> files; single <code>store</code> instance
Repository-like API layer	<code>analyticsApi.ts</code> exposes typed data-access functions abstracting raw HTTP from UI
Builder / Config-map	<code>buildSummaryCards(summary)</code> and <code>STATUS_COLORS/QUIZ_PIE_COLORS</code> lookup maps in <code>SharedCharts</code>
Strategy via role	Conditional chart rendering driven by <code>user.role</code> (<code>isAdmin</code> , <code>isManager</code>)

6. Configuration & Environment

- **Build/dev config:** `vite.config.ts`, `index.html` (SPA entry, though entry point not auto-detected; `main.tsx` mounts the app).
- **TypeScript configs:** `tsconfig.json`, `tsconfig.app.json`, `tsconfig.node.json` (split app vs node tooling).
- **Linting:** `eslint.config.js` (flat config) ignoring `dist`, targeting `**/*.ts,tsx`, ES2020, browser globals.
- **Routing constants:** centralized in `shared/constants/routes.ts` (`ROUTES`).
- **API endpoint constants:** `shared/constants/apiEndpoints.ts` (`API_ENDPOINTS`) -- base URL likely configured inside `apiClient` (not shown).
- **Auth persistence key:** `AUTH_STORAGE_KEY = "auth"` in `authSlice.ts`.
- **Theme:** initialized via `useTheme()` in `App.tsx` on load.

No `.env` references appear in the provided files; environment configuration (API base URL) is presumably handled by Vite env variables inside `apiClient`.

7. Scalability & Deployment Notes

- **Stateless client:** All server state is fetched on demand; only auth (`token/user`) is persisted client-side in `localStorage`. Suitable for **static hosting / CDN** deployment (Vite produces a static `dist/` bundle -- confirmed by ``globalIgnores(['dist'])``)
-

4. Functional Flow

Functional Flow Document -- Project "Frontend" (compliance-track-frontend)

A React + TypeScript + Vite single-page application for compliance tracking, built around Redux Toolkit (auth state), React Router (lazy-loaded routes), and a feature-based folder structure. Backend communication is centralized via an `apiClient`.

1. Application Startup

The startup sequence is inferred from `main.tsx` (entry, not fully shown) -> `App.tsx` -> `AppRouter.tsx`. The Vite entry point is `index.html`, which mounts the app.

Initialization Sequence

Redux store construction -- `src/app/store/store.ts`

```
ts export const store = configureStore({ reducer: { auth: authReducer } });
```

The store is created with a single `auth` slice.

Auth state hydration from storage -- `src/app/store/authSlice.ts`

- The slice's `initialState` is produced by `loadAuthFromStorage()`:

```
ts const initialState: AuthState = loadAuthFromStorage();
```

- `loadAuthFromStorage()` reads `localStorage` key "auth", parses it, and validates that both `token` and `user` are present. On any failure it returns `{ token: null, user: null }`.

Root component mount -- `src/App.tsx`

```
tsx function App() { useTheme(); // initialize theme on app load return (<><Toaster ... /><AppRouter /></>); }
```

- `useTheme()` (from `shared/hooks/useTheme`) initializes the theme.

- The `sonner <Toaster>` is wired globally (top-right, rich colors, 3 visible toasts) -- this is the app-wide notification surface used by `toast.error(...)` / `toast.success(...)` calls throughout features.

Router setup -- `src/app/routes/AppRouter.tsx`

- All feature pages are `lazy()`-loaded and wrapped in `<Suspense fallback={<LoadingFallback />}>`.

- `LoadingFallback()` renders a centered `<LoadingSpinner />`.

- Auth state is read directly from the store:

```
tsx const token = useSelector((state: RootState) => state.auth.token); const user = useSelector((state: RootState) => state.auth.user);
```

Service Wiring

- **API layer:** `shared/api/apiClient` is imported by feature API modules (e.g. `analyticsApi.ts`).

- **Endpoint catalog:** `shared/constants/apiEndpoints` (`API_ENDPOINTS`) centralizes all backend URLs.
- **Route catalog:** `shared/constants/routes` (`ROUTES`) centralizes navigation paths.

2. Main Execution Paths

Path A -- Root Redirect / Role-based Landing

`AppRouter.tsx`:

1. App loads `/`.
2. Router evaluates auth + role:
`tsx element={ token && user?.role ? user.role === "EMPLOYEE" // (truncated, but logic visible) ? <Navigate to={ROUTES.MY_POLICIES}/> : <Navigate to={ROUTES.DASHBOARD}/> : <Navigate to={ROUTES.LOGIN}/> }`
3. **Decision:**
 - No `token` / no `user.role` -> redirect to `ROUTES.LOGIN`.
 - Role `EMPLOYEE` -> redirect to `ROUTES.MY_POLICIES`.
 - Other roles (`ADMIN` / `MANAGER`) -> redirect to `ROUTES.DASHBOARD`.

Path B -- Protected Route Access (PrivateRoute Guard)

`src/app/routes/PrivateRoute.tsx`:

```
export default function PrivateRoute({ allowedRoles }: PrivateRouteProps) { ... }
```

For any guarded route, the following gates run in order:

1. **No token** -> `<Navigate to={ROUTES.LOGIN} replace />`.
2. **No user** (defensive) -> `<Navigate to={ROUTES.LOGIN} replace />`.
3. **No `user.role`** -> `<Navigate to={ROUTES.LOGIN} replace />`.
4. **Role check** (only if `allowedRoles` provided & non-empty):
`tsx const hasRequiredRole = allowedRoles.includes(user.role as UserRole); if (!hasRequiredRole) { const landing = user.role === "EMPLOYEE" ? ROUTES.MY_POLICIES : ROUTES.DASHBOARD; return <Navigate to={landing} replace />; }`
5. **All gates pass** -> `<Outlet />` renders the matched child route inside `MainLayout`.

Path C -- Authenticated Layout Rendering

`src/app/layouts/MainLayout.tsx`:

```
export default function MainLayout() {
  return (
    <div className="d-flex">
      <Sidebar />
      <div className="flex-grow-1">
        <Header />
        <main><Outlet /></main>
      </div>
    </div>
  );
}
```

- Shared by **ADMIN**, **MANAGER**, **EMPLOYEE**.
- `<Outlet />` renders the active feature page (e.g. **DashboardPage**, **MyPolicies**, etc.).

Path D -- Dashboard Data Load (primary analytics use case)

`src/features/analytics/DashboardPage.tsx` is the current dashboard (vs. legacy `DashboardOld.tsx`).

1. Component reads current user from store:
`tsx const user = useSelector((state: RootState) => state.auth.user);`
2. Calls the central analytics hook:
`tsx const analytics = useAnalyticsData({ userId: user?.id ?? 0, role: user?.role ?? "EMPLOYEE", });`
3. **Loading gate**: while `analytics.loading` -> render centered `<LoadingSpinner />`.
4. **Fatal gate**: if `!analytics.summary` -> render Bootstrap `alert-danger` ("Failed to load dashboard summary...").
5. Otherwise compute role flags:
`tsx const isAdmin = user?.role === "ADMIN"; const isManager = user?.role === "MANAGER";`
6. Render `DashboardHeader`, `SharedCharts`, and conditionally `AdminCharts` / `ManagerCharts`.

`useAnalyticsData` load orchestration -- `hooks/useAnalyticsData.ts`

The hook holds a single `AnalyticsState` object and loads data in **3 phases** (per its own documentation):

1. **Phase 1 -- summary** via `getDashboardSummary(userId)`.
2. **Phase 2 -- shared charts**: `getAuditTaskStatus`, `getAverageQuizScores`, `getPoliciesWithQuiz`, `getComplianceTrend`.
3. **Phase 3 -- role-specific charts**:
 - ADMIN: `getMostAssignedPolicies`, `getPoliciesByCategory`, `getDepartmentCompliance`, `getMonthlyRollout`, `getChecklistItemsBubble`.
 - MANAGER: `getTeamQuizHistogram`, `getTeamPendingPolicies`, `getTeamTopPerformers`.

Each call is wrapped in `loadSafely` so a single failure does not block other charts (errors are stored per-key in `state.errors`).

The hook exposes reload functions for single-chart refresh on filter change:

- `reloadTrend(mode, year?)`
- `reloadQuizScores(excludeZero)`
- `reloadTeamHistogram(...)` / `reloadHistogram`
- `reloadTeamTopPerformers(...)` / `reloadTopPerformers`
- `reloadMonthlyRollout(start?, end?)`
- `reloadMostAssigned(top, includeInactive)`

Path E -- Save Dashboard as Report

`DashboardPage.handleSaveReport(data: CreateReportDTO):`

1. Guard: if `!analytics.summary` -> `toast.error("Cannot save report without dashboard summary")` and return.
2. Build a metrics snapshot from all `analytics.*` fields.
3. Call `createReport({ ...data, metrics })` (from `features/reports/reportApi`).
4. Success -> `toast.success("Report saved successfully")`.
5. Failure -> `toast.error(parseError(error))`; re-throw the error.

3. Key Function Call Chains

Auth state lifecycle

```
App startup
-> loadAuthFromStorage()           [authSlice.ts] (reads localStorage "auth")
-> store initialState

Login (LoginPage, lazy-loaded)
-> setAuth(LoginResponse)          [authSlice action]
-> saveAuthToStorage(payload)       [persists to localStorage]

Logout
-> clearAuth()                     [authSlice action]
-> clearAuthStorage()               [removes localStorage "auth"]
```

Dashboard chain

```
DashboardPage
-> useAnalyticsData({ userId, role }) [useAnalyticsData.ts]
  -> loadDashboardData()
    -> getDashboardSummary(userId)     [analyticsApi.ts]
    -> apiClient.get(API_ENDPOINTS.ANALYTICS_SUMMARY)
    -> getAuditTaskStatus(userId)
    -> getAverageQuizScores(userId, excludeZero)
    -> getPoliciesWithQuiz(...)
    -> getComplianceTrend(...)
    -> [ADMIN] getMostAssignedPolicies / getPoliciesByCategory /
              getDepartmentCompliance / getMonthlyRollout /
```

```

        getChecklistItemsBubble
        -> [MANAGER] getTeamQuizHistogram / getTeamPendingPolicies /
        getTeamTopPerformers
    -> DashboardHeader(summary, errorCount, onSaveReport)
    -> SharedCharts(... reloadTrend, reloadQuizScores)
    -> AdminCharts(... reloadMonthlyRollout, reloadMostAssigned) [if isAdmin]
    -> ManagerCharts(... reloadHistogram, reloadTopPerformers) [if isManager]

```

Chart filter -> reload chains

```

AdminCharts.handleRolloutPeriodChange(period)
-> getRolloutDateRange(period) -> { start, end }
-> reloadMonthlyRollout(start, end) [back into useAnalyticsData]
    -> getMonthlyRollout(start, end) -> apiClient.get(ANALYTICS_MONTHLY_ROLLOUT?start=&end=)

AdminCharts.handleTopCountChange(value) / handleIncludeInactiveChange(...)
-> reloadMostAssigned(top, includeInactive)
    -> getMostAssignedPolicies(top, includeInactive)

ManagerCharts.handleHistogramPolicyChange(policyId)
-> reloadHistogram(10, id)

ManagerCharts.handleTopPerfPolicyChange(policyId)
-> reloadTopPerformers(10, 1, id)

SharedCharts.handleTrendModeChange(mode)
-> reloadTrend(mode, mode === "year" ? trendYear : undefined)
SharedCharts.handleExcludeZeroChange(...)
-> reloadQuizScores(excludeZero)

```

Save report chain

```

DashboardHeader (Save button)
---
## 5. Data Flow Analysis
# Data Flow Analysis -- Project "Frontend" (compliance-track-frontend)
> Scope note: This is a React + TypeScript SPA built with Vite. The analysis is grounded in the files actually used in the application.
---
## 1. Input Sources
The application ingests data from the following observable sources:
| Source | Mechanism | Code Reference |
|-----|-----|-----|
| **HTTP REST API** | Axios-based `apiClient` GET/POST calls | `shared/api/apiClient` (imported in `analyticsA
| **Redux store (in-memory)** | `useSelector((state: RootState) => state.auth...)` | `AppRouter.tsx`, `Priv
| **Browser `localStorage`** | Auth token/user persistence | `authSlice.ts` (`AUTH_STORAGE_KEY = "auth&qu
| **User input (forms / controls)** | Dropdowns, toggles, filters | `AdminCharts.tsx`, `ManagerCharts.tsx`, `S
| **URL / Route params** | `react-router` `BrowserRouter`, `Outlet`, `Navigate` | `AppRouter.tsx`, `MainLayout
| **System clock** | `new Date()` for greetings & date-range filters | `DashboardHeader.tsx` (`getGreeting ...

**Key input parameters observed** (passed to API functions):
- `userId: number` -- drives most analytics queries

```

```

- `role: "ADMIN" | "MANAGER" | "EMPLOYEE"` -- controls which data is fetched
- Filter params: `top`, `includeInactive`, `excludeZero`, `binSize`, `policyId`, `minAttempts`, `start`/`end`

```

2. Data Models & Schemas

2.1 Authentication State (`shared/types/auth`, used in `authSlice.ts`)

Type	Field	Notes
`AuthState`	`token: string` \ `null`	JWT/session token
`AuthState`	`user: User` \ `null`	Authenticated user object
`LoginResponse`	`token`, `user`	Returned by login; stored via `saveAuthToStorage`
`User` (inferred from usage)	`id: number`	Used as `userId` in analytics calls
	`role: "ADMIN" \   "MANAGER" \   "EMPLOYEE"`	Used in `PrivateRoute`, role ga ...
	`name: string`	Split for first name in `DashboardHeader`

2.2 Analytics Models (`shared/types/analytics`)

Model	Used By	Key Role
`DashboardSummary`	Summary cards	Fields seen: `overallPolicyCompliancePercent`, `pendingPolicyAcceptance`
`AuditTaskStatusChart`	Shared donut chart	Status buckets PENDING/INPROGRESS/COMPLETED
`AverageQuizScoreResponse`	Shared bar chart	--
`PoliciesWithQuizPieResponse`	Shared pie chart	--
`ComplianceTrendResponse`	Shared line chart	Modes "month"/"year"
`MostAssignedPolicy[]`	Admin chart	Filters: `top`, `includeInactive`
`PoliciesByCategoryResponse`	Admin chart	--
`DepartmentComplianceBar[]`	Admin chart	Admin-only
`MonthlyRollout[]`	Admin chart	Date-range filtered
`ChecklistItemsBubble[]`	Admin chart	--
`TeamQuizHistogram`	Manager chart	Filters: `binSize`, `policyId`
`TeamPendingPolicy[]`	Manager chart + dropdown source	--
`TeamTopPerformer[]`	Manager chart	Filters: `top`, `minAttempts`, `policyId`

2.3 Aggregate UI State (`useAnalyticsData.ts` -- `AnalyticsState`)

`AnalyticsState` holds **all** chart datasets plus:

Field	Type	Purpose
`loading`	`boolean`	Gates initial render
`errors`	`Record<string, string>`	Per-chart error tracking keyed by API name (e.g. `"audit&qu ...`

2.4 Report DTO (`shared/types/report`)

Type	Field	Notes
`CreateReportDTO`	(base fields, not fully shown)	Spread via `...data`
	`metrics`	Snapshot object containing all analytics datasets -- assembled in `handleSaveReport`

3. Transformation Pipeline

3.1 Auth hydration (parse + validate)

`authSlice.ts` -> `loadAuthFromStorage()`:

1. Reads raw string from `localStorage[AUTH\_STORAGE\_KEY]`.
2. `JSON.parse(raw)` -> `AuthState`.
3. **Validation**: if `!parsed?.token || !parsed?.user`, falls back to `{ token: null, user: null }`.
4. All errors caught -> safe default returned. This becomes `initialState`.

3.2 API query construction

In ``analyticsApi.ts``, query strings are built per-endpoint:

- Simple template literals: ``?userId=${userId}`, `?top=${top}&includeInactive=${includeInactive}``.
- ``getMonthlyRollout`` uses ``URLSearchParams``, appending ``start`/`end`` only if present, with ``toISOString(start``

3.3 Filter -> date-range derivation

``AdminCharts.getRolloutDateRange(period)``:

- ``"6months"`` -> ``{ start: new Date(year, month-6, 1), end: now }``
- ``"12months"`` -> 12 months back
- ``"thisyear"`` -> ``{ start: new Date(year,0,1), end: now }``
- default ``"all"`` -> ``{}`` (no params; backend default).

3.4 Presentation transforms

- ``DashboardHeader.buildSummaryCards(summary)`` maps numeric fields to display strings (``.toFixed(1)``, ``String``)
- ``getGreeting()`` maps ``Date().getHours()`` to a greeting string.
- ``firstName = user?.name?.split(" ")[0] ?? ""``.
- Chart components map status codes via ``STATUS_COLORS`` / ``STATUS_LABELS`` constants (``SharedCharts.tsx``), and

3.5 Report snapshot assembly

``DashboardPage.handleSaveReport(data)`` builds a ``metrics`` object by copying every analytics field from the ``an``

4. State Management

4.1 Global state -- Redux Toolkit

- ****Store****: ``app/store/store.ts`` -> ``configureStore({ reducer: { auth: authReducer } })``. Only the ``auth`` ...
- ****Slice****: ``authSlice.ts``
- Reducers: ``setAuth`` (sets ``token`` + ``user`` from ``LoginResponse``), ``clearAuth`` (nulls both).
- Side-effect helpers (not reducers): ``saveAuthToStorage``, ``clearAuthStorage``, ``loadAuthFromStorage`` -- brid
- ****Selectors****: read-only access via ``useSelector`` in ``AppRouter``, ``PrivateRoute``, ``DashboardPage``, ``Dashboar``

4.2 Feature/local state -- React hooks

- ****`useAnalyticsData` hook**** holds the entire dashboard dataset in a single ``useState<AnalyticsState>`` ...
- ****Component-local filter state**** (``useState``):
- ``AdminCharts``: ``rolloutPeriod``, ``topCount``, ``includeInactive``
- ``ManagerCharts``: ``histogramPolicyFilter``, ``topPerfPolicyFilter``
- ``SharedCharts``: ``trendMode``, ``trendYear``, ``excludeZero``
- ``DashboardPage``: ``showReportModal``
- ****Theme state****: ``useTheme()`` hook invoked in ``App.tsx`` (initializes theme on load -- implementation in ``sha``

5. Output Destinations

Destination	What is written	Code Reference
localStorage	Auth <code>`{token, user}`</code> JSON under key <code>`"auth"`</code>	<code>`saveAuthToStorage`</code> / <code>`clearAu ...`</code>
***HTTP POST (backend)**	New report with metrics snapshot	<code>`createReport(...)`</code> in <code>`reportApi`</code> , called from
***UI / DOM**	Charts (recharts), summary cards, tables	All <code>`*Charts.tsx`</code> , <code>`DashboardHeader.tsx`</code>
***Toast notifications (side effect)**	Success/error feedback	<code>`sonner`</code> <code>`toast.success/.error`</code> , <code>`<Toast ...`</code>
***Navigation (side effect)**	Route redirects	<code>`<Navigate>`</code> in <code>`PrivateRoute`</code> , <code>`AppRouter`</code>

Error output flows through ``parseError(error)`` (``shared/utils/errorParser``) -> ``toast.error(...)``.

6. API Request/Response Flows

All analytics requests go through ``apiClient.get<T>(url)`` and resolve to ``response.data``. Endpoints are ...

Function	Method	Endpoint Constant	Query Params	Returns	Access
`getDashboardSummary`	GET	`ANALYTICS_SUMMARY`	`userId`	`DashboardSummary`	All
`getMostAssignedPolicies`	GET	`ANALYTICS_MOST_ASSIGNED_POLICIES`	`top`, `includeInactive`	`MostAssignedPolicies`	All
`getPoliciesByCategory`	GET	`ANALYTICS_POLICIES_BY_CATEGORY`	`userId`	`PoliciesByCategoryResponse`	All
`getDepartmentCompliance`	GET	`ANALYTICS_COMPLIANCE_DEPARTMENT`	--	`DepartmentComplianceBar[]`	Admin
`getMonthlyRollout`	GET	`ANALYTICS_MONTHLY_ROLLOUT`	`start?`, `end?`		

6. Component Relationships

Component Relationship Map -- Frontend (compliance-track-frontend)

> **Scope Note:** This analysis is based on the provided files. ~87 KB of source was truncated, so portions ..

1. Module Dependency Graph

1.1 Application Bootstrap & Routing Layer

main.tsx [inferred]

+> App.tsx

+> ./App.css

+> sonner (Toaster)

+> shared/hooks/useTheme

+> app/routes/AppRouter.tsx

+> react (lazy, Suspense)

+> react-router (BrowserRouter, Routes, Route, Navigate)

+> react-redux (useSelector)

+> shared/constants/routes (ROUTES)

+> shared/ui/LoadingSpinner

+> shared/ui/PlaceholderPage

+> shared/ui/NotFoundPage

+> app/store/store (RootState)

+> app/routes/PrivateRoute

+> [lazy] feature pages:

+> features/auth/LoginPage

+> features/auth/RegisterPage

+> app/layouts/MainLayout

+> features/analytics/DashboardPage

+> features/master-data/MasterDataPage

+> features/reports/ReportsPage

+> features/user-management/usermanagementTab

+> features/policy/AdminPolicyList

+> features/policyAssessment/PolicyAssessmentTab

+> features/policyAssessment/Quiz

+> features/policyAssessment/ManagerAndEmployeeQuiz/TakeQuiz

+> features/compliance/ComplianceAdminTab

- +> features/compliance/ComplianceManagerTab
- +> features/auditTask/admin/AdminAuditTaskPage
- +> features/auditTask/manager/ManagerAuditTaskPage
- +> features/auditTask/employee/EmployeeAuditTaskPage
- +> features/audit-task-checklist/AuditChecklistPage
- +> features/audit-task-checklist/CreateAuditChecklistPage
- +> features/my-policies/MyPolicies
- +> features/user-dashboard/UserDashboard

1.2 State Management Layer

app/store/store.ts

- +> app/store/authSlice.ts (authReducer, default export)
- +> shared/types/auth (AuthState, LoginResponse)

PrivateRoute.tsx --> app/store/store (RootState)

- +> shared/constants/routes (ROUTES)

AppRouter.tsx --> app/store/store (RootState)

1.3 Layout Layer

app/layouts/MainLayout.tsx

- +> react-router (Outlet)
- +> shared/components/Sidebar [inferred]
- +> shared/components/Header [inferred]

1.4 Analytics Feature (most complete subtree shown)

features/analytics/DashboardPage.tsx

- +> react-redux (useSelector)
- +> sonner (toast)
- +> app/store/store (RootState)
- +> features/analytics/hooks/useAnalyticsData
- +> features/analytics/components/DashboardHeader
- +> features/analytics/components/SharedCharts
- +> features/analytics/components/AdminCharts
- +> features/analytics/components/ManagerCharts
- +> features/analytics/components/ChartComponents (SectionTitle)
- +> shared/utils/errorParser (parseError)
- +> shared/ui/LoadingSpinner
- +> features/reports/CreateReportModal

+> features/reports/reportApi (createReport)
+> shared/types/report (CreateReportDTO)

features/analytics/hooks/useAnalyticsData.ts
+> react (useState, useEffect)
+> shared/utils/errorParser (parseError)
+> features/analytics/analyticsApi (15 getters)
+> shared/types/analytics (14 types)

features/analytics/analyticsApi.ts
+> shared/api/apiClient
+> shared/constants/apiEndpoints (API_ENDPOINTS)
+> shared/utils/dateFormatter (toISOString)
+> shared/types/analytics

features/analytics/components/AdminCharts.tsx
+> recharts
+> features/analytics/components/ChartComponents
| (ErrorCard, EmptyCard, ChartCard, ChartRow, ChartColumn)
+> shared/utils/formatNumber
+> shared/types/analytics

features/analytics/components/ManagerCharts.tsx
+> recharts
+> features/analytics/components/ChartComponents
+> shared/utils/formatNumber
+> shared/types/analytics

features/analytics/components/SharedCharts.tsx
+> recharts
+> features/analytics/components/ChartComponents
+> shared/utils/formatNumber
+> shared/types/analytics

features/analytics/components/DashboardHeader.tsx
+> lucide-react
+> react (useMemo)
+> react-redux (useSelector)
+> app/store/store (RootState)
+> shared/types/analytics (DashboardSummary)
+> ../dashboard.css

features/analytics/components/ChartComponents.tsx
+> react (ReactNode)
+> lucide-react

features/analytics/DashboardOld.tsx [LEGACY -- not routed]
+> react, react-redux, sonner, lucide-react, recharts

+-> app/store/store (RootState)
+-> features/analytics/analyticsApi
+-> shared/types/analytics
+-> shared/utils/errorParser
+-> shared/ui/LoadingSpinner
+-> features/reports/CreateReportModal
+-> features/reports/reportApi (createReport)
+-> shared/types/report

features/analytics/DashboardDemo.tsx [STUB -- no imports, not routed]

1.5 Cross-Feature Edge

features/analytics/ --> features/reports/CreateReportModal

features/analytics/ --> features/reports/reportApi

This is the only **direct feature-to-feature** dependency visible (analytics -> reports).

2. Class Hierarchy

The codebase is **React + TypeScript functional** in style. There are **no ES class inheritance trees** (``class``

Interface (declared as "class")	File	Role	Extends
---	---	---	---
`PrivateRouteProps`	PrivateRoute.tsx	Component props	--
`AdminChartsProps`	AdminCharts.tsx	Component props	--
`ManagerChartsProps`	ManagerCharts.tsx	Component props	--
`SharedChartsProps`	SharedCharts.tsx	Component props	--
`SummaryCardConfig`	DashboardHeader.tsx	View-model config	--
`DashboardHeaderProps`	DashboardHeader.tsx	Component props	--
`AnalyticsState`	useAnalyticsData.ts	Hook state shape	--
`UseAnalyticsDataOptions`	useAnalyticsData.ts	Hook input	--

> **Finding:** No inheritance. Composition and prop-drilling are the structural mechanisms. This is idiomat ...

3. Composition Relationships

Since there are no runtime classes, "composition" maps to **component containment** and **hook owner** ...

Owner / Container	Owns / Renders	Relationship
---	---	---
`App`	`Toaster`, `AppRouter`	renders; calls `useTheme()`
`AppRouter`	`PrivateRoute`, all lazy feature pages, `LoadingFallback`	route composition
`MainLayout`	`Sidebar`, `Header`, ` <outlet></outlet> `	persistent shell
`DashboardPage`	`useAnalyticsData` (hook), `DashboardHeader`, `SharedCharts`, `AdminCharts`, `ManagerChart`	
`useAnalyticsData`	15 `analyticsApi` functions	data orchestration (3-phase load + per-chart reloaders)
`AdminCharts`	`ErrorCard`, `EmptyCard`, `ChartCard`, `ChartRow`, `ChartColumn`	+ recharts primitives
`ManagerCharts`	same `ChartComponents` set	+ recharts chart composition
`SharedCharts`	same `ChartComponents` set	+ recharts chart composition
`DashboardHeader`	`buildSummaryCards()`	-> `SummaryCardConfig[]` builds view-models from `DashboardSu ...

```
| `store` | `authReducer` | Redux store composition (single slice: `auth`) |
```

```
### Data-flow ownership of state
```

DashboardPage

| owns: showReportModal (local)

| consumes: useSelector(state.auth.user)

+ - useAnalyticsData(userId, role)

owns: AnalyticsState (summary, charts, errors, loading)

exposes: reloadTrend, reloadQuizScores, reloadTeamHistogram,

reloadTeamTopPerformers, reloadMonthlyRollout, reloadMostAssigned

| +- passed DOWN as props to:

+ -> AdminCharts (reloadMonthlyRollout, reloadMostAssigned)

+ -> ManagerCharts (reloadHistogram, reloadTopPerformers)

+ -> SharedCharts (reloadTrend, reloadQuizScores)

Filter UI state (e.g. `rolloutPeriod`, `topCount`, `histogramPolicyFilter`) is **localized in the child ...

```
---
```

4. Shared Utilities & Helpers

```
| Shared module | Consumed by (observed) | Purpose |
```

```
| ---|---|---|
```

```
| `shared/api/apiClient` | `analyticsApi` (and presumably every `*Api`) | HTTP client base |
```

```
| `shared/constants/apiEndpoints` (`API_ENDPOINTS`) | `analyticsApi` | endpoint registry |
```

```
| `shared/constants/routes` (`ROUTES`) | `AppRouter`, `PrivateRoute` | route constants |
```

```
| `shared/utls/errorParser` (`parseError`) | `DashboardPage`, `DashboardOld`, `useAnalyticsData` | normalize
```

```
| `shared`
```

```
---
```

7. File Inventory

```
| File | Language | Lines | Classes | Functions | Entry Point |
```

```
| -----|-----|-----:|-----:|-----:|:-----:|
```

```
| `compliance-track-frontend\eslint.config.js` | JavaScript | 23 | 0 | 0 | |
```

```
| `compliance-track-frontend\src\App.tsx` | TSX | 24 | 0 | 1 | |
```

```
| `compliance-track-frontend\src\app\layouts\MainLayout.tsx` | TSX | 21 | 0 | 1 | |
```

```
| `compliance-track-frontend\src\app\routes\AppRouter.tsx` | TSX | 161 | 0 | 2 | |
```

```
| `compliance-track-frontend\src\app\routes\PrivateRoute.tsx` | TSX | 51 | 1 | 1 | |
```

```
| `compliance-track-frontend\src\app\store\authSlice.ts` | TypeScript | 65 | 0 | 3 | |
```

```
| `compliance-track-frontend\src\app\store\store.ts` | TypeScript | 12 | 0 | 0 | |
```

```
| `compliance-track-frontend\src\features\analytics\DashboardDemo.tsx` | TSX | 9 | 0 | 1 | |
```

```
| `compliance-track-frontend\src\features\analytics\DashboardOld.tsx` | TSX | 484 | 0 | 4 | |
```

```
| `compliance-track-frontend\src\features\analytics\DashboardPage.tsx` | TSX | 166 | 0 | 2 | |
```

```
| `compliance-track-frontend\src\features\analytics\analyticsApi.ts` | TypeScript | 175 | 0 | 13 | |
```

```
| `compliance-track-frontend\src\features\analytics\components\AdminCharts.tsx` | TSX | 436 | 1 | 5 | |
```

```
| `compliance-track-frontend\src\features\analytics\components\ChartComponents.tsx` | TSX | 90 | 0 | 7 | |
```

```
| `compliance-track-frontend\src\features\analytics\components\DashboardHeader.tsx` | TSX | 130 | 2 | 3 | |
```

```
| `compliance-track-frontend\src\features\analytics\components\ManagerCharts.tsx` | TSX | 279 | 1 | 4 | |
```

```
| `compliance-track-frontend\src\features\analytics\components\SharedCharts.tsx` | TSX | 379 | 1 | 5 | |
```

```
| `compliance-track-frontend\src\features\analytics\hooks\useAnalyticsData.ts` | TypeScript | 284 | 2 | 8 | |
```

```
| `compliance-track-frontend\src\features\audit-task-checklist\AuditChecklistPage.tsx` | TSX | 300 | 0 | 8 | |
```

```
| `compliance-track-frontend\src\features\audit-task-checklist\CreateAuditChecklistPage.tsx` | TSX | 144 | 0 | |
```

```
| `compliance-track-frontend\src\features\audit-task-checklist\EditAuditChecklistModal.tsx` | TSX | 86 | 1 | 3 |
```

`compliance-track-frontend\src\features\audit-task-checklist\auditChecklistApi.ts`	TypeScript	51	0	5
`compliance-track-frontend\src\features\auditTask\AuditTaskApis.ts`	TypeScript	36	0	2
`compliance-track-frontend\src\features\auditTask\Search.tsx`	TSX	24	0	1
`compliance-track-frontend\src\features\auditTask\admin\AdminAssignedTask.tsx`	TSX	111	0	2
`compliance-track-frontend\src\features\auditTask\admin\AdminAuditTaskPage.tsx`	TSX	159	0	4
`compliance-track-frontend\src\features\auditTask\employee\EmployeeAuditTaskApi.tsx`	TSX	31	0	3
`compliance-track-frontend\src\features\auditTask\employee\EmployeeAuditTaskPage.tsx`	TSX	163	0	3
`compliance-track-frontend\src\features\auditTask\employee\EmployeeViewAuditTaskModal.tsx`	TSX	137	1	1
`compliance-track-frontend\src\features\auditTask\manager\EditAuditTaskModal.tsx`	TSX	243	1	6
`compliance-track-frontend\src\features\auditTask\manager\ManagerAuditTaskApi.ts`	TypeScript	116	0	11
`compliance-track-frontend\src\features\auditTask\manager\ManagerAuditTaskPage.tsx`	TSX	259	0	7
`compliance-track-frontend\src\features\auditTask\manager\ViewAuditTaskModal.tsx`	TSX	94	1	2
`compliance-track-frontend\src\features\auth\LoginPage.tsx`	TSX	96	0	2
`compliance-track-frontend\src\features\auth\RegisterPage.tsx`	TSX	257	0	5
`compliance-track-frontend\src\features\auth\authApi.ts`	TypeScript	37	0	3
`compliance-track-frontend\src\features\compliance\ComplianceAdminAPIs.ts`	TypeScript	17	0	1
`compliance-track-frontend\src\features\compliance\ComplianceAdminTab.tsx`	TSX	109	0	3
`compliance-track-frontend\src\features\compliance\ComplianceManagerAPIs.ts`	TypeScript	34	0	3
`compliance-track-frontend\src\features\compliance\ComplianceManagerTab.tsx`	TSX	170	0	4
`compliance-track-frontend\src\features\compliance\CreateAuditTaskModal.tsx`	TSX	237	1	6
`compliance-track-frontend\src\features\employee-type\employeeTypeApi.ts`	TypeScript	50	0	4
`compliance-track-frontend\src\features\master-data\MasterDataPage.tsx`	TSX	70	0	1
`compliance-track-frontend\src\features\master-data\modals\DeleteConfirmModal.tsx`	TSX	87	1	3
`compliance-track-frontend\src\features\master-data\modals\DeleteDepartmentModal.tsx`	TSX	76	1	3
`compliance-track-frontend\src\features\master-data\modals\EditDepartmentModal.tsx`	TSX	125	1	3
`compliance-track-frontend\src\features\master-data\modals\EditEmployeeTypeModal.tsx`	TSX	108	1	3
`compliance-track-frontend\src\features\master-data\modals\EditPolicyCategoryModal.tsx`	TSX	137	1	3
`compliance-track-frontend\src\features\master-data\modals\ViewDepartmentModal.tsx`	TSX	60	0	2
`compliance-track-frontend\src\features\master-data\modals\ViewPolicyCategoryModal.tsx`	TSX	44	0	1
`compliance-track-frontend\src\features\master-data\tabs\DepartmentTab.tsx`	TSX	279	0	8

8. API Endpoints

No API endpoints were detected in the analyzed files.

9. Dependencies

No dependency file found. Inferred imports from source files:

- ../auditTask.css
- ../dashboard.css
- ./App.css
- ./auditTask.css
- AdminAssignedTask
- AppRouter
- Button
- CreateAuditTaskModal
- CreateReportModal
- DeleteConfirmModal
- DeleteDepartmentModal
- DepartmentTab
- EditAuditChecklistModal
- EditDepartmentModal

EmployeeTypeTab
EmployeeViewAuditTaskModal
Header
Input
LoadingSpinner
PlaceholderPage
PolicyCategoryTab
PrivateRoute
React
Sidebar
ViewAuditTaskModal
ViewDepartmentModal
apiClient
authReducer
bootstrap/dist/css/bootstrap.min.css
dayjs
globals
js
reactHooks
reactRefresh
tseslint
type
...

10. Developer Onboarding Guide

Developer Onboarding Guide -- Frontend

Note on detected metadata: The scan did not detect config/dependency files (e.g., `package.json`), build entry points, or a lockfile. This guide is written for the detected stack (JavaScript, TypeScript, TSX) but flags every place where you must verify against the actual repository, since automated detection returned no concrete config. **Do not assume commands work until you confirm them against the files described below.**

1. Prerequisites

Because no `package.json` or lockfile was detected, you must first determine the toolchain by inspecting the repo. The detected stack (`.ts`, `.tsx`, `.js`) indicates a TypeScript + JSX (React-family) frontend. Based on that:

1. **Node.js** -- required for any TypeScript/TSX frontend.
 - Verify the expected version:

```
bash cat .nvmrc 2>/dev/null || cat .node-version 2>/dev/null
```

- If neither exists, install the current LTS (Node 20.x) as a safe default:

```
bash node --version
```

2. **A package manager** -- confirm which one is in use by checking for a lockfile:

```
bash ls package-lock.json yarn.lock pnpm-lock.yaml bun.lockb 2>/dev/null
```

The lockfile present tells you whether to use **npm**, **yarn**, **pnpm**, or **bun**.

3. **Git** -- for cloning and branching.

4. **A code editor with TypeScript + JSX support** (VS Code recommended) so **.tsx** type errors surface inline.

5. **Accounts/secrets** -- check for an env template before assuming none are needed:

```
bash ls .env.example .env.local.example .env.sample 2>/dev/null
```

If one exists, you'll need credentials for whatever services it references (API keys, auth providers, etc.).

2. Setup Steps

1. **Clone the repository:**

```
bash git clone <repo-url> cd <repo-directory>
```

2. **Confirm the project root.** Since no entry point or **package.json** was detected at the top level, the actual app may live in a subdirectory (e.g., **app/**, **src/**, **frontend/**, or a monorepo **packages/***). Find it:

```
bash find . -name package.json -not -path '*/node_modules/*'
```

cd into the directory containing the relevant **package.json**.

3. **Read the **scripts** block** of that **package.json** to learn the real commands:

```
bash cat package.json
```

This is the authoritative source for build/run/test commands.

4. **Install dependencies** using the package manager matched to the lockfile from Prerequisites step 2:

```
bash npm install # if package-lock.json # or: yarn install # or: pnpm install # or: bun install
```

5. **Set up environment variables** if a template exists:

```
bash cp .env.example .env.local
```

Fill in required values.

3. Running the Project

The exact commands below are placeholders **until you read the **scripts** field** (Setup step 3). Frontend TS/TSX projects commonly expose these script names; substitute the real ones.

- **Start the dev server:**

```
bash npm run dev # common (Vite, Next.js) # or: npm start # common (Create React App)
```

- **Type-check** (TypeScript):

```
``bash
```

